

Lecture 1: RAG Fundamentals & Architecture Overview

Your AI Engineering Teacher — 10+ Years Experience

Part of the RAG Mastery Series for AI/ML Engineer Preparation

What You Will Learn Today

1. What is RAG and why does it matter?
 2. The complete RAG architecture — every component explained
 3. Naive RAG vs Advanced RAG vs Agentic RAG
 4. Real-world applications of RAG
 5. Why companies are hiring RAG engineers
-

1. What is RAG? (The Simple Explanation)

RAG = Retrieval-Augmented Generation

Think of it like this:

You're writing an exam. You have a **closed-book exam** (LLM alone — it only knows what it was trained on) vs an **open-book exam** (RAG — the LLM can look up information before answering).

RAG lets an LLM look up relevant information from YOUR data before generating an answer.

The Problem RAG Solves

Problem	LLM Alone	With RAG
Hallucination (making up facts)	❌ Common	✅ Reduced
Out-of-date knowledge	❌ Stuck at training cutoff	✅ Always current

Domain-specific questions	❌ Generic answers	✅ Accurate, specific
No access to private data	❌ Can't see your files	✅ Uses your documents
No source citation	❌ Can't prove answers	✅ Shows sources

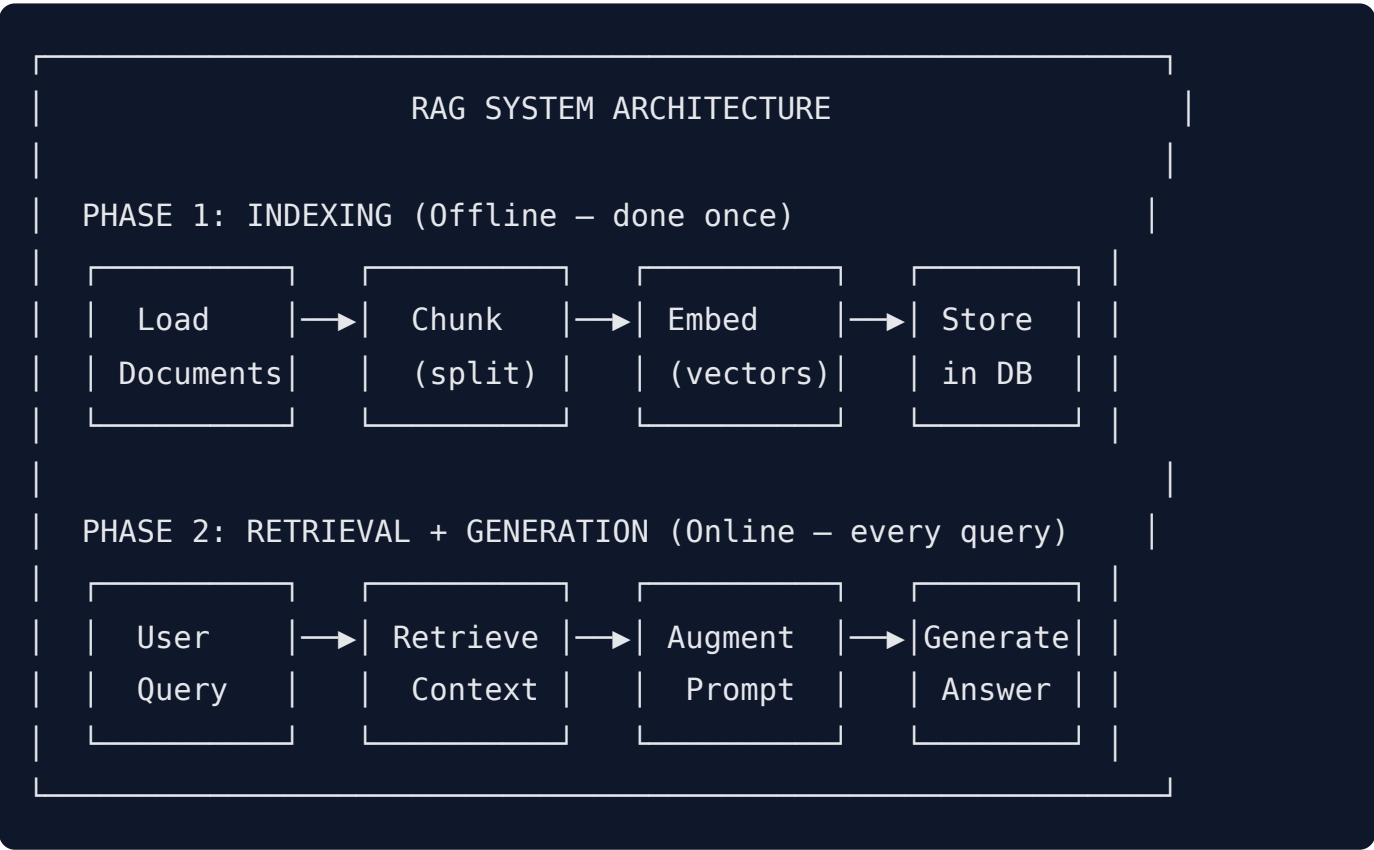
A Simple Analogy

Imagine you're a customer support agent:

- **Without RAG:** You answer from memory. You might give wrong info about the latest product.
- **With RAG:** You search the company knowledge base first, find the exact policy, then answer confidently.

2. The Complete RAG Architecture

Here's the full pipeline, step by step:



Phase 1: Indexing (Building the Knowledge Base)

Step 1: Document Loading

Load documents from various sources:

- PDFs, Word docs, text files
- Web pages (HTML)
- Databases
- APIs (Notion, Confluence, Slack)
- Images (with OCR)

```
# Example: Loading a PDF
from langchain.document_loaders import PyPDFLoader
loader = PyPDFLoader("my_document.pdf")
documents = loader.load()
```

Step 2: Chunking (Splitting Documents)

Break large documents into smaller pieces called **chunks**.

Why chunk? LLMs have limited context windows. You can't feed a 100-page PDF into an LLM. So you split it into manageable pieces and only retrieve the relevant ones.

Chunking Strategies:

Strategy	Chunk Size	Overlap	Best For
Fixed-size	512 tokens	50-100 tokens	Simple documents
Recursive	Variable	Yes	Mixed content
Semantic	By meaning	No	Topic-based
Document-based	By section	No	Structured docs
Agentic	LLM decides	No	Complex docs

```
# Example: Recursive Chunking
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=50
)
chunks = splitter.split_documents(documents)
```

Step 3: Embedding (Converting Text to Numbers)

Convert each chunk into a **vector** (a list of numbers) using an **embedding model**.

What is a vector? It's a list of numbers (e.g., 1536 numbers) that represents the MEANING of the text. Similar meanings have similar vectors.

```
"cat"      → [0.23, -0.45, 0.89, ...] (1536 numbers)
"kitten"   → [0.25, -0.42, 0.91, ...] (similar = close together)
"car"      → [-0.89, 0.12, -0.34, ...] (different = far apart)
```

Popular Embedding Models:

- `text-embedding-3-small` (OpenAI) — 1536 dimensions, cheap
- `all-MiniLM-L6-v2` (Sentence Transformers) — free, 384 dimensions
- `BGE-large` (BAAI) — open source, very good
- `E5-large` (Microsoft) — strong multilingual

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(["This is a chunk of text"])
```

Step 4: Vector Storage

Store vectors in a **Vector Database** for fast similarity search.

Popular Vector Databases:

- **ChromaDB** — Easy to start, good for prototyping
- **Pinecone** — Managed, production-ready

- **Weaviate** — Open source, hybrid search
- **Qdrant** — Fast, Rust-based, open source
- **Milvus** — Enterprise scale
- **FAISS** — Facebook's library (in-memory, not a DB)
- **pgvector** — PostgreSQL extension (use existing DB)

Phase 2: Retrieval + Generation (Answering Questions)

Step 5: Query Processing

When a user asks a question:

1. Convert the query into a vector (same embedding model)
2. Search the vector DB for the most similar chunks

```
User: "What is the refund policy?"  
    ↓  
Query Vector: [0.12, -0.34, 0.56, ...]  
    ↓  
Search Vector DB → Find top-K similar chunks  
    ↓  
Return: [chunk_42, chunk_7, chunk_15, ...]
```

Step 6: Retrieval Strategies

Dense Retrieval (Vector Search):

- Uses embedding similarity (cosine similarity)
- Good for semantic matching
- "What is ML?" matches "Machine Learning explained"

Sparse Retrieval (BM25 / Keyword Search):

- Uses keyword matching (TF-IDF based)
- Good for exact terms, names, codes
- "Python 3.11 release date" matches exact keywords

Hybrid Retrieval (Best of Both):

- Combines dense + sparse
- Uses a reranker to merge results

- This is what production systems use in 2025-2026

```
# Hybrid Search Example
from langchain.retrievers import EnsembleRetriever
retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.4, 0.6] # 40% keyword, 60% semantic
)
```

Step 7: Reranking (Optional but Important)

After retrieving top-K chunks, a **reranker** reorders them by relevance.

Why? The first retrieval is fast but approximate. Reranking is slower but more accurate.

Popular Rerankers:

- `bge-reranker-v2-m3` (BAAI) — open source, excellent
- `Cohere Rerank` — API-based, very good
- `ColBERT` — token-level interaction, very accurate

Step 8: Prompt Augmentation

Combine the retrieved chunks with the user's query into a prompt:

```
System: You are a helpful assistant. Use the context below to answer.
```

```
Context:
```

```
[Chunk 42]: Our refund policy allows returns within 30 days...
```

```
[Chunk 7]: Refunds are processed within 5-7 business days...
```

```
[Chunk 15]: Digital products are non-refundable after download...
```

```
User Question: What is the refund policy?
```

```
Answer:
```

Step 9: Generation

Send the augmented prompt to the LLM, which generates the final answer using the retrieved context.

3. Three Generations of RAG

Naive RAG (Basic — What Your Course Taught)

```
Query → Embed → Vector Search → Top-K Chunks → Prompt → LLM → Answer
```

Problems:

- Retrieves irrelevant chunks sometimes
- No quality control
- Can't handle complex questions
- Single retrieval only

Advanced RAG (Production-Grade — What You Need to Learn)

```
Query → Query Rewriting → Hybrid Search → Reranking →  
Context Compression → Prompt → LLM → Answer
```

Improvements:

- Pre-processing: query rewriting, expansion, decomposition
- Better retrieval: hybrid search, multi-hop, recursive
- Post-processing: reranking, filtering, compression
- Handles long documents better

Agentic RAG (State of the Art — 2025-2026)

```
Query → Agent decides:  
  → Search web? Search DB? Search multiple DBs?  
  → Need to refine query?  
  → Retrieved enough? Or search again?  
  → Synthesize from multiple sources  
  → Generate answer with citations
```

Key Features:

- LLM agent decides WHEN and WHAT to retrieve

- Can use multiple tools (web search, SQL, APIs)
- Iterative retrieval (search → evaluate → search again)
- Self-correction (if results are bad, try differently)

4. Real-World RAG Applications (2025-2026)

Industry	Application	Companies
Healthcare	Medical diagnosis assistance, patient record search	Epic, Tempus
Legal	Contract analysis, case law search	Harvey AI, CoCounsel
Finance	Report analysis, compliance checking	Bloomberg GPT, JP Morgan
Customer Support	Automated support with company knowledge	Intercom, Zendesk
Education	Personalized tutoring, textbook Q&A	Khan Academy, Duolingo
Software Eng	Code search, documentation Q&A	GitHub Copilot, Cursor
Enterprise	Internal knowledge management	Glean, Notion AI
E-commerce	Product search, recommendations	Amazon, Shopify
Research	Paper search, literature review	Consensus, Semantic Scholar

5. Key Metrics to Evaluate RAG Systems

Metric	What It Measures	Good Value
Faithfulness	Does the answer match the retrieved context?	> 0.9
Answer Relevancy	Does the answer address the question?	> 0.85
Context Precision	Are the retrieved chunks relevant?	> 0.7

Context Recall	Did we retrieve all needed info?	> 0.8
Latency	How fast is the response?	< 2 seconds

Evaluation Frameworks: RAGAS, TruLens, DeepEval

6. Quick Summary

RAG = Retrieve relevant docs + Augment prompt + Generate answer

Indexing Pipeline: Load → Chunk → Embed → Store

Query Pipeline: Query → Retrieve → Rerank → Augment → Generate

Three Types: Naive → Advanced → Agentic

Key Technologies: Embedding Models, Vector DBs, Rerankers, LLMs

Homework

1. Set up a basic RAG system using LangChain + ChromaDB + any free LLM
2. Load 3 PDFs and ask questions about them
3. Try different chunk sizes (256, 512, 1024) and observe the difference
4. Compare BM25 search vs vector search vs hybrid search

Resources to Explore

- LangChain RAG Tutorial: <https://python.langchain.com/docs/tutorials/rag/>
- LlamaIndex Docs: <https://docs.llamaindex.ai/>
- RAGAS Evaluation: <https://docs.ragas.io/>
- Pinecone Learning Center: <https://www.pinecone.io/learn/>

Next Lecture: Vector Databases Deep Dive — How they work internally, HNSW algorithm, and choosing the right one for your project.